Language Features under Weight Quantization

Activation Agreement and Intervention Effects

Hemang Joshi Eashaan Thakur Shardul Joshi Nikhita Ravi

IIIT Hyderabad – ANLP Monsoon 2026

The Research Question we started with

When a model's weights are rounded to fewer bits,

do the internal features that explain it

still do the same job?

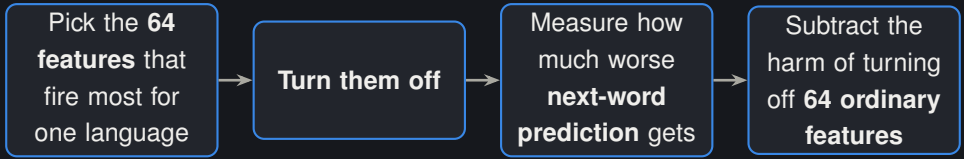
27 languages · Gemma-2-2B (main) · Gemma-2-9B check · a fixed sparse autoencoder (SAE)

Background

- **Quantization** stores weights with fewer bits ($8 \ldots 4$). Cheaper, but behaviour can change.
- A **sparse autoencoder** splits a hidden state into a few active **features**.
- Some features fire mostly for **one language**.
- Earlier work asked only: does a feature still *fire* the same way after quantization?

We also ask: does it still do the same thing?

Methodology



Repeat on the full model (FP) and with weights rounded to **8, 7, 6, 5 and 4 bits**

The result is the **excess harm**: how much those features matter for that language.

A correction to earlier work

	No BOS	With BOS
FP perplexity	458.4	16.1
INT6 survival	51%	97.5%
INT7 survival	81%	>99.9%

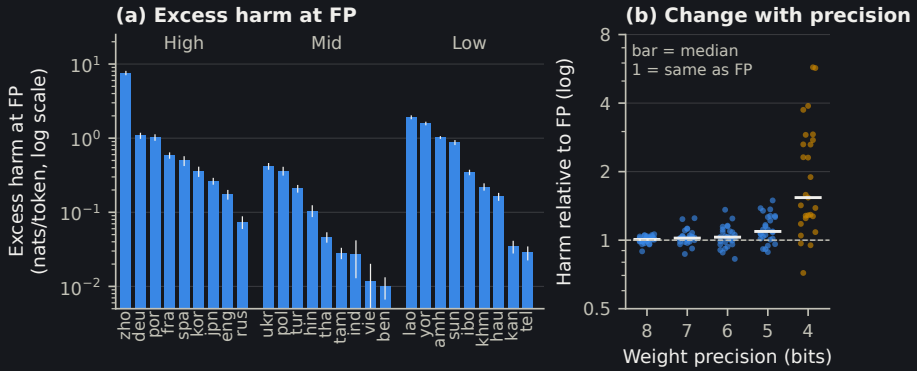
We re-ran the earlier study's own code and matched its numbers.

Its inputs left out the **start-of-text token** (BOS).

That one token made the model look 28× worse and half the features look "dead".

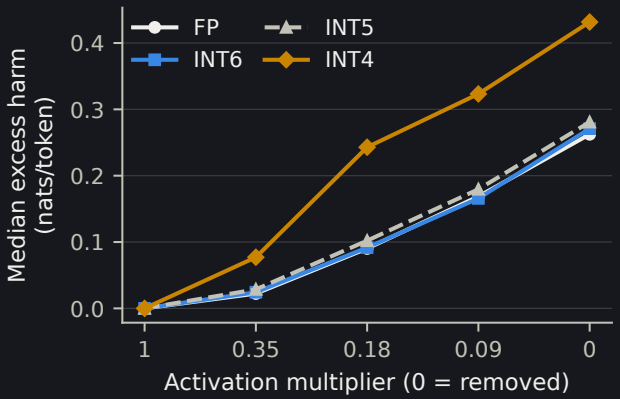
All our results use the corrected inputs. We also retired our own first pipeline after finding flaws in it.

Finding 1: language features matter, in every language



Left: turning off a language's features hurts that language in all 27. **Right:** the harm stays close to full precision down to 5 bits, then grows at 4 bits.

Finding 2: more suppression, more harm



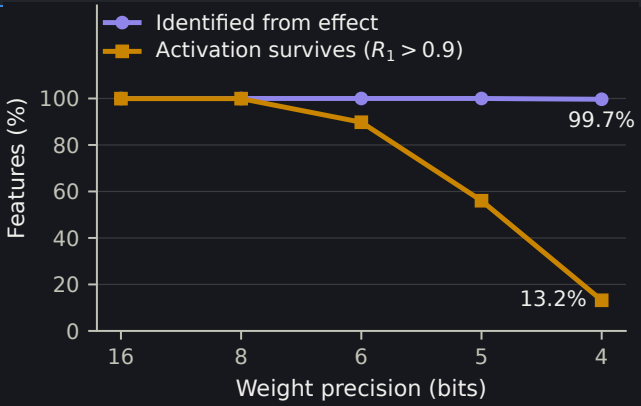
We turned features **down** step by step instead of fully off.

Harm rises smoothly, in every language.

At 4 bits the same step hurts more.

A graded response is evidence that these features have a real causal role.

Finding 3: “dead” features still work

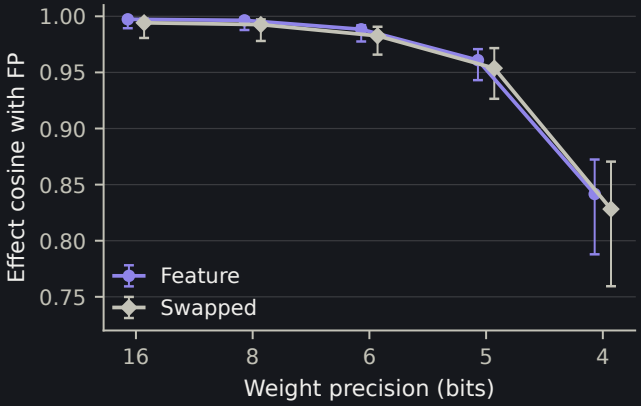


Orange: the usual test says only 13% of features survive at 4 bits.

Violet: yet 99.7% can still be recognised from what they do to the output.

Whether a feature *fires the same* and whether it *works the same* are different questions.

Finding 4: the change is not about the feature

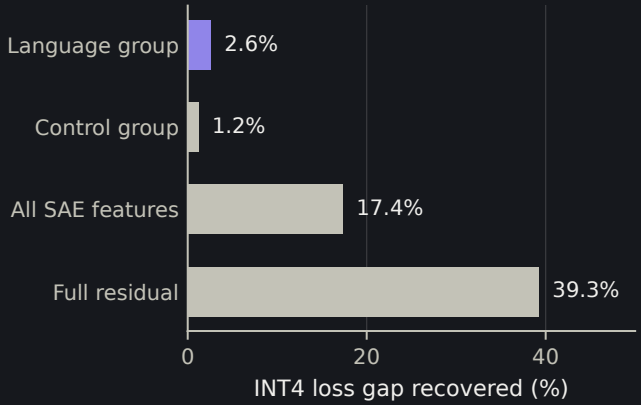


How similar is a feature's effect to its full-precision effect?

A **random other direction** at the same places drifts almost equally.

The 4-bit drift comes from the model around the features, not from the features.

Finding 5: fixing the features fixes little



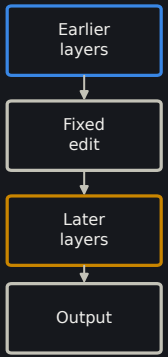
We put the full-precision values back into the 4-bit model.

Restoring the language features recovers only 2.6% of the lost quality.

Restoring the whole layer recovers 39%.

The features are not where the compression damage sits.

Finding 6: where does the change in an edit come from?



C: only these are quantized: the edit meets a new input

Same feature, same strength as at FP

W: only these are quantized: the edit is processed differently

I: extra change when both are quantized

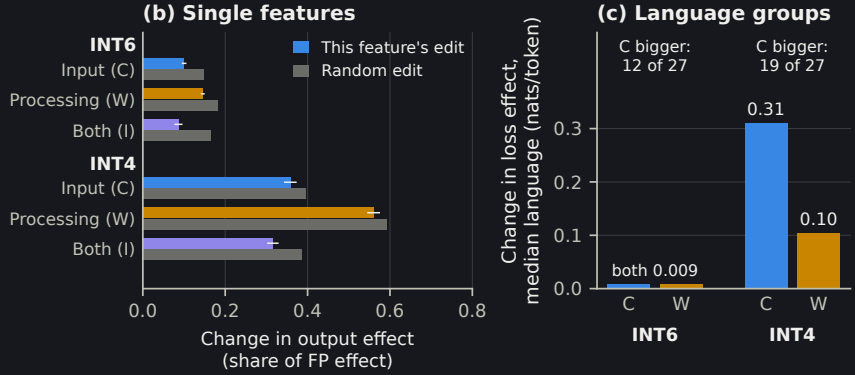
$D[\text{early}, \text{late}]$ = how much the edit changes the output, with the early and late layers each at full precision (FP) or 4-bit (Q).

$$C = D[\text{Q}, \text{FP}] - D[\text{FP}, \text{FP}]$$
$$W = D[\text{FP}, \text{Q}] - D[\text{FP}, \text{FP}]$$
$$I = D[\text{Q}, \text{Q}] - D[\text{Q}, \text{FP}] - D[\text{FP}, \text{Q}] + D[\text{FP}, \text{FP}]$$

$C + W + I$ adds up exactly to the full change $D[\text{Q}, \text{Q}] - D[\text{FP}, \text{FP}]$.

We keep one edit exactly as at full precision and round only part of the model.

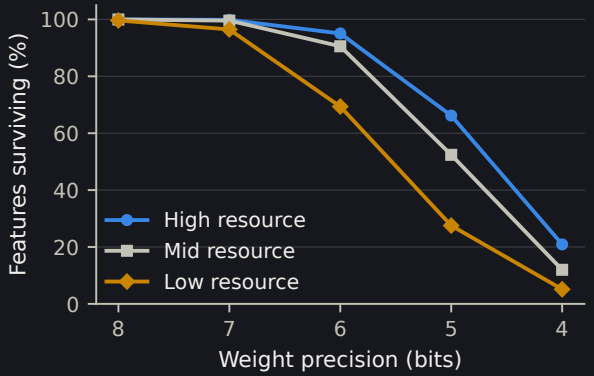
Finding 6: the answer depends on what we measure



One feature: processing (W) matters most; a random edit changes as much.

A language's group: the new input (C) matters most (19 of 27 at 4 bits).

Which features stop firing? Weak ones.



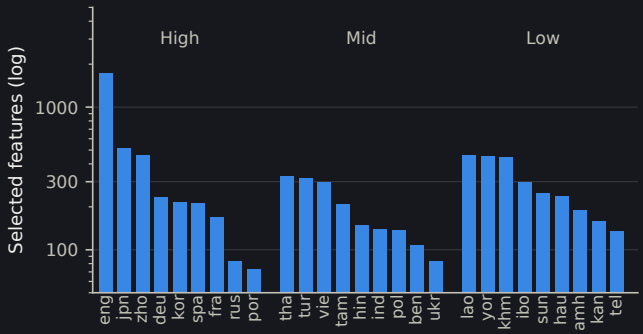
Low-resource languages lose more features.

Language-selective features stop firing slightly more often.

Both gaps come from weaker activations: at equal strength they disappear.

Activation strength, not language or resource level, predicts which features stop firing.

More features $\neq$ more resources



Lao, Yoruba and Khmer have more language-specific features than German or Spanish.

A feature counts when it fires for one language much more than the rest.

Close relatives (Spanish–Portuguese, Russian–Ukrainian) share features.

The count shows how distinct a language looks to the model, not its resources (descriptive).

Does it hold elsewhere?

Gemma-2-9B

Same picture: harm in 27/27 languages, about the same at 6 bits, growth at 4 bits.

All predictions written *before* the run were met.

Three quantizers

RTN as in prior work, standard RTN and GPTQ all give the same pattern.

Charts show prior work's RTN for direct comparison.

Llama-3.2-1B

Weaker: harm in 21/27 languages (rule: 24).

Model, SAE and tokenizer all differ, so the cause is unclear.

Limitations

- **Reading comprehension (Belebele):** turning the features off does not change accuracy. Our held-out, confirmatory test. The features matter for next-word prediction, not clearly for this task.
- **One layer, one SAE:** earlier work finds language-specific units mostly in early and late layers; we tested only a middle layer (compute budget), a conservative place to look.
- **Fixed-length inputs:** sentences are cut to a set length. The same cut text is used at every precision, so comparisons stay fair; testing complete sentences is future work.
- **Mostly exploratory:** most FLORES analyses reuse sentences we had already examined.

Claims and Findings

1. Language features help next-word prediction in all 27 languages, on both Gemma sizes.
2. Turning them down step by step gives a smooth dose-response.
3. The usual activation test understates preservation: 13% “survive”, 99.7% still work.
4. The 4-bit drift is generic, and restoring the features repairs only 2.6%.
5. Activation agreement, task performance and an edit’s effect need separate measurements.

Reproducibility

- Reproduced the earlier study before reporting its BOS problem
- Predictions and code hashes frozen before the data existed
- Two faulty 9B predictions documented, not counted
- Null and failed results reported alongside the positive ones

Code: `github.com/TH4KUR/always-late`
Artifacts: `huggingface.co/datasets/karma-skz/always-late-artifacts`
Runs: `wandb.ai/karma-skz-/anlp-project`